A Mini Decoder-Only Transformer Language Model from Scratch: Design, Training, and a Character vs. BPE Tokenization Study

Ibrahem H. Ali*[a,1] and Kevin O. Bonsu†[a,1]

[a] *Department of Computer Science, Franklin and Marshall College*

Abstract—We built a decoder-only transformer from scratch and trained it on the Shakespeare corpus under two tokenization schemes: character-level (65-token vocabulary) and Byte Pair Encoding (BPE, 1,000 tokens). Both models are identical in architecture (∼10.8M parameters, 6 layers, 6 attention heads, 384-dimensional embeddings) and training setup—the tokenizer is the only thing we changed. The character-level model converges to a validation perplexity of **3.53**. The BPE model hits 49.90 at step 1,500 then falls apart, ending at 119.89. The gap comes down to vocabulary size: 1,000 BPE tokens is too many for a 1.1M-character Shakespeare corpus, and the model overfits to rare merge tokens it cannot generalize. On small literary corpora, character-level tokenization is the better choice.

1. INTRODUCTION

Tokenization rarely gets treated as a real design decision. Most papers pick a tokenizer and move on. But on small corpora the choice matters a lot: it sets vocabulary size, sequence length, and how much signal the model actually sees per step.

We built a GPT-style decoder-only transformer from scratch in PyTorch and trained it on Shakespeare (∼1.1M characters) under two tokenization regimes: a character-level tokenizer (65 tokens) and a custom BPE tokenizer (1,000 tokens) trained on the same corpus. Everything else—architecture, optimizer, learning rate, batch size—is identical.

The result is clear. Character-level wins by a large margin, not because the architecture changes but because a 1,000-token BPE vocabulary is a poor fit for Shakespeare's narrow lexicon. We trace the failure to three concrete causes and report full training curves for both models.

The project also includes a FastAPI backend with per-token confidence scoring (Shannon entropy), multi-head attention visualization, and a web interface for interactive generation.

2. MODEL ARCHITECTURE

We use a standard decoder-only transformer. Table 1 lists the hyperparameters, which are the same for both tokenization runs.

Table 1. Architecture hyperparameters (same for both models).

Hyperparameter	Value
Layers	6
Attention heads	6
Embedding dim	384
Head dimension	64
Context window	256 tokens
FFN expansion	4× (384→1536→384)
FFN activation	GELU
Dropout	0.2
Params (char)	∼10.8M
Params (BPE)	∼11.1M

The small parameter difference comes from the BPE embedding table (1,000×384 vs. 65×384).

Each block applies attention then a feedforward network, both with residual connections and pre-norm:

$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{MHA}(\mathrm{LN}(\mathbf{x})), \quad \mathbf{x} \leftarrow \mathbf{x} + \mathrm{FFN}(\mathrm{LN}(\mathbf{x})). \tag{1}$$

*iali@fandm.edu
†koseibon@fandm.edu

Equal Contribution. Ibrahem designed the dashboard, model architecture, tokenization pipeline, and API server. Kevin trained both models, debugged and tested the code, implemented entropy confidence, ran the experiments, created the graphs, and wrote the documentation. All other parts of the project were completed collaboratively by Ibrahem and Kevin.

Placing LayerNorm *before* each sub-layer (pre-norm) keeps gradients well-scaled throughout training [1]. Causal masking enforces the autoregressive property:

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V, \tag{2}$$

where M sets future positions to $-\infty$.

Three other choices worth noting: the token embedding and output head share weights (weight tying), attention projections have no bias, and positional embeddings are learned absolute vectors added before the first block.

3. TOKENIZATION

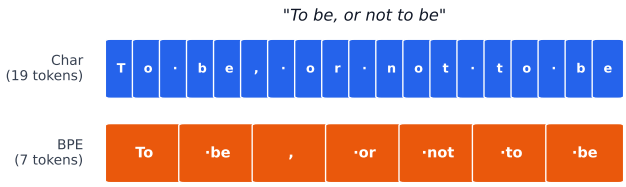

Figure 1. Character-level produces one token per character; BPE merges characters into subwords.

3.1. Character-Level

Each of the 65 unique characters in Shakespeare gets a fixed integer index: letters (upper and lower), digits, punctuation, space, and newline. Encoding is a dictionary lookup—no training needed, no information lost.

3.2. Byte Pair Encoding (BPE)

We implement BPE from scratch [2]. Starting from the same 65 characters, the algorithm repeatedly finds the most frequent adjacent pair and merges it into a new token. After 935 merges, vocabulary size reaches 1,000.

Note

BPE compresses better: ∼3.8 chars/token vs. ∼1 for char-level, so each 256-token window covers ∼973 Shakespeare characters. But the corpus shrinks to ∼290K BPE tokens vs. ∼1.1M character tokens—fewer training examples at the same step count.

4. TRAINING SETUP

Both models train under the same configuration (Table 2).

Table 2. Training hyperparameters (same for both models).

Setting	Value
Optimizer	AdamW
Peak LR	3.0×10^{-4}
Min LR	3.0×10^{-5}
LR warmup	100 steps (linear)
LR schedule	Cosine annealing
Batch size	64
Training iterations	5,000
Weight decay	0.1
Gradient clipping	1.0 (max norm)
Train/val split	90% / 10%
Eval interval	Every 500 steps

The loss is cross-entropy averaged over all positions:

$$\mathcal{L} = -\frac{1}{BT}\sum_{b=1}^{B}\sum_{t=1}^{T}\log p_\theta\left(x_{t+1}^{(b)} \mid x_{\le t}^{(b)}\right), \tag{3}$$

where $B = 64$ and $T = 256$. Learning rate follows a 100-step linear warmup then cosine decay:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(1 + \cos(\pi\alpha))(\eta_{\max} - \eta_{\min}), \tag{4}$$

where $\alpha = (t - t_{\text{warm}})/(t_{\text{decay}} - t_{\text{warm}})$. Gradient clipping at max norm 1.0 keeps training stable. The best checkpoint (lowest val loss) is saved after every evaluation, with periodic snapshots every 1,000 steps.

5. EXPERIMENTS AND RESULTS

5.1. Results

Table 3 shows the final numbers. Tables 4 and 5 give the full training progression.

Table 3. Final results for both tokenization strategies.

Model	Vocab	Best Val Loss	Final PPL
Char-level	65	**1.2600**	**3.53**
BPE	1,000	3.9099†	119.89

†*BPE best at step 1,500; val loss then climbs to 4.7866.*

Table 4. Character-level model: val loss and perplexity.

Step	Val Loss	PPL
0	4.2053	67.04
500	1.9687	7.16
1,000	1.5857	4.88
2,000	1.3611	3.90
3,000	1.2894	3.63
4,000	1.2648	3.54
4,999	**1.2600**	**3.53**

Table 5. BPE model: train loss, val loss, and perplexity. Val loss starts rising at step 1,500 while train loss keeps falling.

Step	Train Loss	Val Loss	PPL
0	6.9875	6.9876	1,083
500	4.0761	4.3723	79.2
1,000	3.4215	3.9410	51.5
1,500	2.9873	**3.9099**	**49.9**
2,000	2.5441	4.0242	55.9
3,000	1.9709	4.3962	81.1
4,000	1.7132	4.6576	105.4
4,999	1.6209	4.7866	119.9

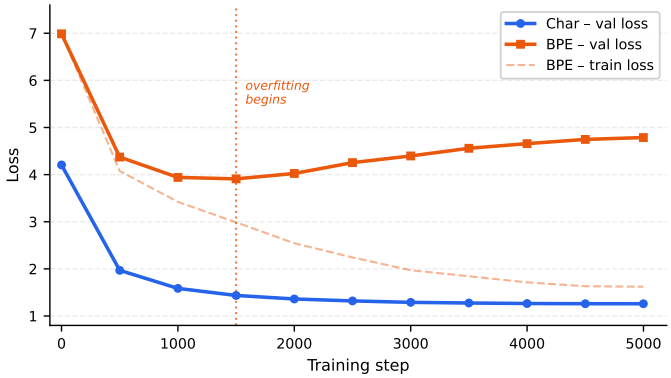

Figure 2. Validation loss over 5,000 steps. Char converges; BPE overfits after step 1,500.

5.2. Analysis

The char model behaves exactly as expected. Train and val loss both fall throughout, with a final gap of just 0.10 nats. Perplexity drops 94.7% from initialization (67.04) to the final checkpoint (3.53).

The BPE model is a different story. Train loss falls the entire time, reaching 1.62 by step 4,999. But val loss turns around at step 1,500 and climbs all the way to 4.79—a train/val gap of 3.17 nats, roughly 30× wider than the char model. The model is memorizing the training set.

Three things push BPE toward overfitting here:

1. **Vocabulary too large for the corpus.** 1,000 tokens is a lot for Shakespeare. Many of the 935 merge tokens show up rarely in training, so the 384,000-weight output projection is hard to regularize with the same dropout and weight decay that works fine for 65 tokens.
2. **Not enough diversity to generalize merges.** BPE merge rules learn from co-occurrence patterns. Shakespeare's vocabulary is narrow and repetitive—there just is not enough variety for 935 merge rules to hold up on held-out text.
3. **Fewer tokens, fewer updates.** BPE compression shrinks the corpus from ∼1.1M to ∼290K tokens. At the same step count, the BPE model sees fewer unique contexts.

6. SYSTEM DESCRIPTION

We also built a full inference system around the trained models.

Text generation. Generation uses temperature scaling and top-k filtering. Temperature τ rescales logits ($\tilde{z}_i = z_i/\tau$): lower values produce more predictable output, higher values more varied. Top-k drops everything outside the k most likely tokens before sampling. We use $\tau = 0.8$ and $k = 40$ by default.

Per-token confidence scoring. Every generated token gets a Shannon entropy score:

$$H_t = -\sum_{v} p_\theta(v \mid x_{\le t}) \log p_\theta(v \mid x_{\le t}). \tag{5}$$

Low entropy means the model was confident; high entropy means it was guessing. This gives token-level interpretability without any external evaluation tools.

Attention visualization. After each forward pass, we store the full attention weight matrices from all 6 layers and 6 heads. These 6×6 maps (shape $T \times T$ each) are exposed via API so you can see which positions the model attended to for any predicted token.

Web interface. A FastAPI backend handles generation, model switching (char or BPE), attention extraction, and training log retrieval. A small HTML/JS frontend lets you interact with the model directly in the browser.

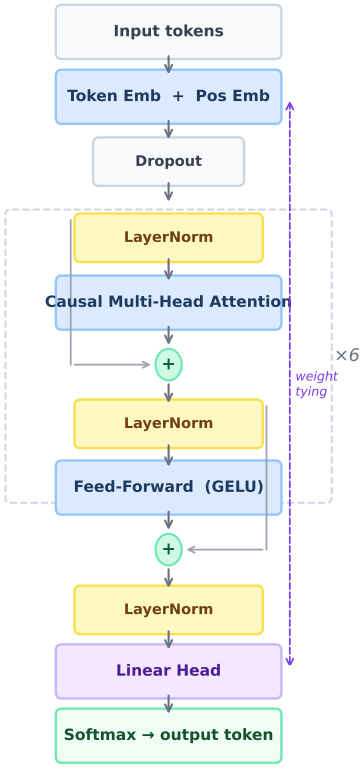

Figure 3. Decoder-only transformer architecture. Each block runs pre-norm attention then FFN, both with residual connections, repeated ×6. The token embedding and linear head share weights (weight tying).

7. DISCUSSION

Character-level works well here because the corpus is small and narrow. Every character contributes a gradient, the 65-token vocabulary is easy to regularize, and the model never hits a token it has barely seen. BPE's advantage—picking up on morphological and subword patterns— only pays off when the corpus is large enough for those patterns to generalize. Shakespeare is not that corpus.

BPE starts winning at scale. At 32K–100K tokens trained on web-scale data, subword tokenization reliably beats character-level [3, 2]. The crossover probably sits somewhere between 1M and 10M training tokens, depending on how diverse the text is. For a 1.1M-character literary corpus, character-level is just the better tool.

Limitations. We only tested one BPE vocabulary size (1,000). A smaller vocab, say 100–300 tokens, might fit Shakespeare much better. We also did not try a pretrained tokenizer, and we stopped training at 5,000 steps.

Future Work

- Sweep BPE vocab sizes (100, 200, 500, 1,000) to find what fits this corpus.
- Test on a larger, more diverse dataset like Project Gutenberg, where BPE should close the gap.
- Compare with SentencePiece unigram tokenization [4].

8. CONCLUSION

We trained a ∼10.8M parameter decoder-only transformer on Shakespeare under two tokenization schemes. Character-level hits perplexity 3.53 and converges cleanly. BPE stalls at 49.9, then overfits to 119.89—34× worse. The models are architecturally identical; the difference is entirely in the tokenizer. A 1,000-token BPE vocabulary is simply too large for a 1.1M-character corpus, and the model cannot regularize it. If you are training on a small literary dataset, use character-level tokenization—it is simpler and it works.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the ACL*, 2016.

[3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *Proceedings of EMNLP: System Demonstrations*, 2018.

[5] A. Karpathy. The unreasonable effectiveness of recurrent neural networks. Blog post, 2015. http://karpathy.github.io/2015/05/21/rnn-effectiveness/

[6] A. Karpathy. nanoGPT. GitHub repository, 2022. https://github.com/karpathy/nanoGPT